

Automated Cloud Deployment Pipeline

Capstone Project Report

Containerized Web Application on AWS with Terraform, GitHub Actions CI/CD
and CloudWatch Monitoring

Student: [Your Name — Student Number]

Module / Course: [Module Name / Code]

Institution: [Institution]

Date: September 2026

GitHub repository: <https://github.com/SUB-Bzero/Automated-Cloud-Deployment-Pipeline>

Table of Contents

1. Introduction
 2. System Architecture
 3. Implementation Steps
 4. Evidence — Screenshots of Executed Work
 5. GitHub Repository
 6. Cost Estimate and Teardown
 7. Conclusion
- Appendix A — Configuration Reference
- Appendix B — Command Reference
- Appendix C — AWS Deployment Runbook (step by step)

1. Introduction

1.1 Project overview

This capstone project delivers a complete automated cloud deployment pipeline. A containerized Node.js/Express web application is deployed to Amazon Web Services: the infrastructure is provisioned with Terraform (Infrastructure as Code), the build and release process is automated with GitHub Actions (CI/CD), and the running system is monitored with AWS CloudWatch alarms that send email alerts through Amazon SNS.

The application exposes a /health endpoint that reports application and database status, a small REST API persisted in Amazon RDS PostgreSQL, and a status home page that shows the deployed version. Public traffic reaches the application through an Internet-facing Application Load Balancer (ALB) that continuously health-checks the container; the database sits in private subnets with no direct Internet exposure.

1.2 Objectives (from the project brief)

1. Write a multi-stage Dockerfile producing a lightweight application container.
2. Use docker-compose to test the Web App + Database locally.
3. Provision modular Terraform infrastructure: VPC with public/private subnets and Security Groups; an EC2 instance to host the container; an RDS PostgreSQL database; and an S3 bucket for safe remote Terraform state.
4. Configure a .github/workflows/deploy.yml pipeline triggered on git push: linting & testing; build & push to AWS ECR; infrastructure deploy via terraform apply; and application deploy to the cloud host.
5. Configure /health endpoints and CloudWatch alarms that email an alert when CPU utilization exceeds 80% or the HTTP 5xx error rate spikes, with rollback capability.

1.3 Tools, platforms and services

Layer	Technology
Application	Node.js 22, Express 4, pg (PostgreSQL client)
Testing & linting	node:test (built-in runner), supertest, ESLint 9
Containerization	Docker multi-stage build, Docker Compose
Infrastructure as Code	Terraform 1.9 + AWS provider 5.x (modular layout)
CI/CD	GitHub Actions (deploy.yml, bootstrap.yml)
AWS services	VPC, EC2, ALB, RDS PostgreSQL, ECR, S3, DynamoDB, SNS, CloudWatch, SSM Parameter Store, IAM
Local verification DB	PostgreSQL (postgres:16-alpine in docker-compose and CI; an embedded PostgreSQL 18 server for the development/test database in this workspace)

2. System Architecture

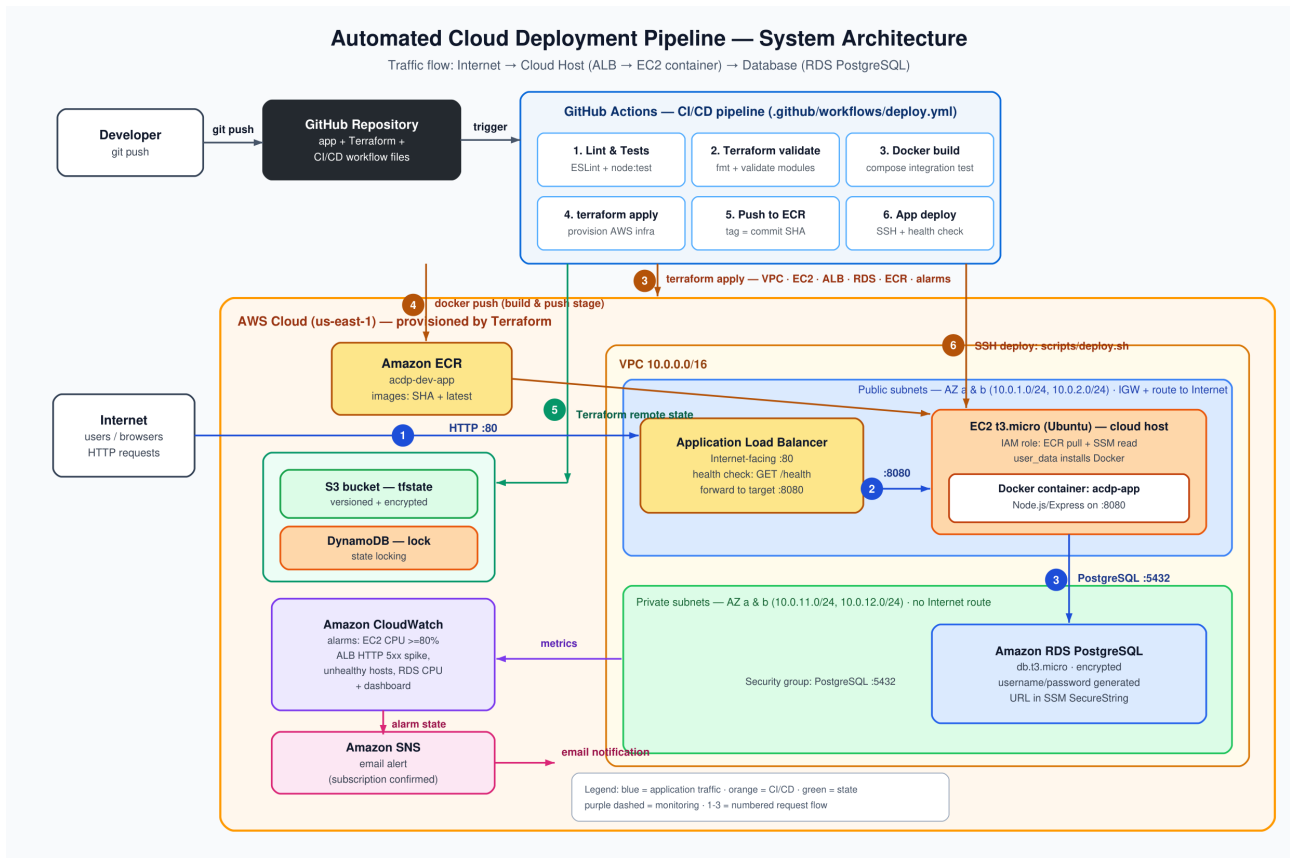


Figure 1: System architecture — traffic flows Internet → ALB → EC2 container (cloud host) → RDS PostgreSQL; the GitHub Actions pipeline provisions the infrastructure, pushes images to ECR and deploys over SSH; CloudWatch alarms notify by email.

2.1 Traffic flow (Internet → Cloud Host → Database)

1. A user on the Internet sends an HTTP request to the application URL (http://) on port 80.
2. The request reaches the Internet-facing Application Load Balancer located in the public subnets of the VPC (behind an Internet Gateway).
3. The ALB forwards the request to the registered EC2 target on port 8080, where the Docker container runs the Node.js/Express application. The ALB continuously probes GET /health and only routes to healthy targets.
4. The application queries the Amazon RDS PostgreSQL instance over port 5432. RDS lives in the private subnets and accepts connections only from the application security group.
5. Responses flow back through the same path to the user.

2.2 Component responsibilities

Component	Responsibility
ALB (public subnets)	Single public entry point; terminates HTTP :80; /health health checks; source of HTTP 5xx metrics for alarmig.
EC2 t3.micro (public subnet)	Cloud host running Docker; pulls the application image from ECR using its IAM instance role; no credentials stored on the host.

Component	Responsibility
RDS PostgreSQL (private subnets)	Managed database; storage encrypted; not publicly accessible; connection URL stored in SSM Parameter Store as a SecureString.
ECR	Private container registry; images tagged with the git commit SHA and latest; scan-on-push and lifecycle policy enabled.
S3 + DynamoDB (state backend)	Versioned, encrypted Terraform remote state; DynamoDB table prevents concurrent applies (state locking).
CloudWatch + SNS	Four metric alarms (EC2 CPU, ALB 5xx, unhealthy hosts, RDS CPU) plus a dashboard; alarms email the subscribed address.
GitHub Actions	CI/CD orchestration: lint, test, validate, build, provision, push and deploy — described in Section 3.3.

2.3 Key security decisions

- Security groups are chained: the ALB SG accepts HTTP only from the Internet; the app SG accepts application traffic only from the ALB SG; the database SG accepts PostgreSQL only from the app SG.
- RDS sits in private subnets with no route to the Internet.
- The database password is generated by Terraform (`random_password`) and never appears in the repository; the full connection URL is stored in SSM Parameter Store (`SecureString`) and read by the EC2 instance through a least-privilege IAM role.
- The EC2 instance requires IMDSv2 (`metadata_options http_tokens = required`).
- The container runs as a non-root user inside a slim runtime image.

3. Implementation Steps

3.1 Application containerization

What was done: a Node.js/Express application with a PostgreSQL-backed items API, a /health probe and a status page was developed and packaged with a multi-stage Dockerfile; a docker-compose.yml file boots the Web App and the database together for local testing.

Main steps followed:

1. Implemented the application (Express app factory, pg connection pool, idempotent schema migration, graceful shutdown) with configuration taken from environment variables (DATABASE_URL, APP_VERSION, PORT).
2. Implemented the /health endpoint to return HTTP 200 with a live database check, and 503 when the database is unreachable — this drives the Docker HEALTHCHECK, the ALB target-group health check and the CI smoke test.
3. Wrote 8 unit/integration tests (node:test + supertest) covering healthy and unhealthy /health responses, the items API, validation and 404 handling.
4. Wrote the multi-stage Dockerfile: stage 1 (builder) installs production dependencies with npm ci --omit=dev; stage 2 (runtime) copies only node_modules and source into node:22-alpine, creates a non-root user, defines the HEALTHCHECK and runs node server.js.
5. Wrote docker-compose.yml with a postgres:16-alpine service (health-checked with pg_isready) and the app service that waits for database health before starting.
6. Verified locally: docker compose up --build --wait, then curl http://localhost:8080/health (this same verification also runs automatically in CI — see Section 4).

Tools used: Node.js 22, Express, pg, node:test, supertest, ESLint, Docker, Docker Compose, PostgreSQL 16.

Purpose of key configurations: the multi-stage build keeps the runtime image small (no build tooling, no dev dependencies) and faster/cheaper to pull on the EC2 host; running as a non-root user limits container compromise impact; the HEALTHCHECK lets `docker compose up --wait` (and the CI job) block until the app is actually ready; the database retry loop in server.js tolerates RDS being slow to accept connections on first boot.

3.2 Infrastructure as Code (Terraform)

What was done: a modular Terraform codebase was written that provisions the complete AWS environment, with a separate bootstrap module that creates the remote state backend (S3 + DynamoDB) before anything else exists.

Terraform module	Provisions
terraform/bootstrap	S3 bucket for state (versioning, AES-256 encryption, public access blocked) + DynamoDB lock table — applied once per account.
modules/network	VPC (10.0.0.0/16), Internet Gateway, 2 public subnets (ALB + EC2), 2 private subnets (RDS), route tables, and the three chained security groups (ALB / app / db).
modules/ecr	ECR repository with scan-on-push, mutable tags and a lifecycle policy (expire untagged images after 7 days, keep the 20 most recent).

Terraform module	Provisions
modules/database	RDS PostgreSQL (db.t3.micro, 20 GB gp3, encrypted, 7-day backups, private subnets), random master password, and the connection URL written to SSM Parameter Store (SecureString).
modules/compute	IAM role + instance profile (ECR read, SSM parameter read), EC2 instance bootstrapped by user_data (installs Docker, prepares /etc/acdp/env), Internet-facing ALB, target group with GET /health checks, HTTP listener.
modules/monitoring	SNS topic + email subscription, four CloudWatch metric alarms and a CloudWatch dashboard.
envs/dev	Root module wiring everything together with a partial S3 backend; concrete backend values are injected at terraform init time.

Main steps followed:

1. Created the bootstrap module and applied it once to create the state bucket (named -acdp-tfstate) and the acdp-tf-locks DynamoDB table.
2. Wrote the reusable modules (network → ecr → database → compute → monitoring) with clearly typed variables and outputs.
3. Wired the modules together in terraform/envs/dev and committed terraform.tfvars with the environment values (instance classes, key pair name, alarm email).
4. Validated the code continuously in CI with terraform fmt -check and terraform validate on every module (see Section 4.2).
5. Deployment itself is performed by the CI/CD pipeline (terraform apply -auto-approve), so infrastructure and application changes are always applied through the same audited path.

Tools used: Terraform 1.9, AWS provider 5.x, S3/DynamoDB backend.

Purpose of key configurations: remote state in a versioned, encrypted S3 bucket protects the state file (which is sensitive and can be restored if accidentally damaged); the DynamoDB lock table prevents two concurrent applies from corrupting state; the partial backend block in backend.tf lets the identical code target any account/region by passing -backend-config at init time; random_password + SSM SecureString keeps database credentials out of the repository and off the instance disk.

3.3 Continuous Integration & Deployment (GitHub Actions)

What was done: .github/workflows/deploy.yml implements the full pipeline, triggered on every git push; .github/workflows/bootstrap.yml provides the one-time backend setup as a manual workflow.

Stage (job)	Runs	Purpose
Preflight	always	Detects whether AWS/SSH secrets are configured and writes a pipeline summary.
1. Lint & Tests	always	ESLint + node:test against a real PostgreSQL service container.
2. Terraform fmt & validate	always	Formatting check plus init/validate of every module — no AWS account needed.
3. Docker build & compose test	always	Builds the multi-stage image and boots app + database with docker compose up --wait, then curls /health.
4. Infrastructure Deploy	with AWS secrets	terraform init (S3 backend) + terraform apply; exports ALB DNS, EC2 IP and ECR URL.

Stage (job)	Runs	Purpose
5. Build & Push to ECR	with AWS secrets	Builds the image, tags it with the commit SHA and latest, pushes to ECR.
6. App Deploy	with AWS + SSH secrets	Copies scripts/deploy.sh to the EC2 host over SSH, pulls the new image, swaps the container, verifies /health through the ALB, and automatically rolls back to the previous image if the health check fails.

Ordering rationale: the infrastructure stage runs before the image push because the ECR repository must exist before an image can be pushed to it (a chicken-and-egg problem if the registry is created by Terraform). The image build itself is still validated up-front in stage 3, so a broken Dockerfile fails fast.

Purpose of key configurations: the concurrency group ensures only one deployment per branch runs at a time; the preflight secret probe keeps the pipeline green (AWS stages skipped, not failed) until credentials are attached, which allowed the CI stages to be verified independently; images are tagged with the git commit SHA so every deployment is traceable to an exact code revision; the deploy script records the previous tag on the host to make rollback a one-command operation.

3.4 Monitoring & rollback

What was done: four CloudWatch metric alarms were provisioned, an SNS email subscription for alert delivery, a CloudWatch dashboard, and rollback mechanisms at both the pipeline and host level.

Alarm	Condition (period / evaluation)	Meaning
acdp-dev-ec2-cpu-high	CPUUtilization > 80% (5 min × 2)	The cloud host is saturated.
acdp-dev-alb-5xx-errors	HTTPCode_Target_5XX_Count sum > 5 (5 min × 1)	The application is returning HTTP 5xx errors at a spiking rate.
acdp-dev-alb-unhealthy-hosts	UnHealthyHostCount > 0 (1 min × 3)	The /health probe is failing — the app is down.
acdp-dev-rds-cpu-high	CPUUtilization > 80% (5 min × 2)	The database is saturated.

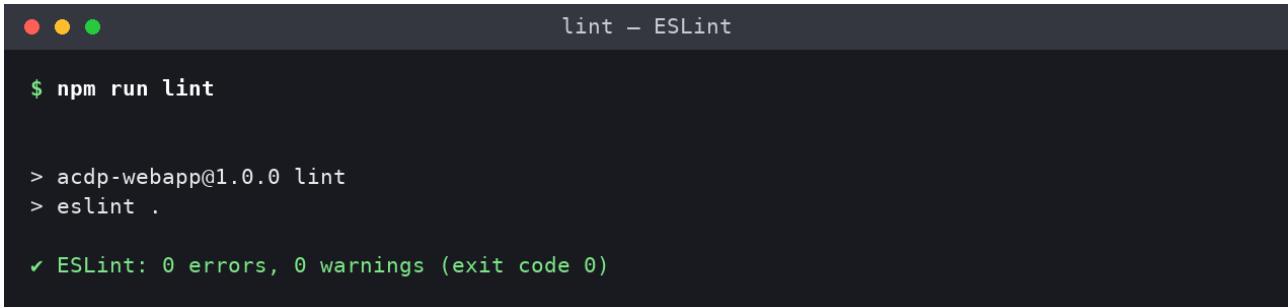
All alarms send email through the SNS topic (the subscription must be confirmed once from the inbox). The dashboard (acdp-dev-dashboard) shows the four corresponding graphs.

Rollback paths: (1) automatic — if the post-deploy health check through the ALB fails, the pipeline re-runs deploy.sh with the previous image tag and fails the job; (2) manual — running the workflow with image_tag = previous (or any commit SHA) redeploys an earlier version. The deploy script keeps the current and previous tags on the host in /opt/acdp.

4. Evidence — Screenshots of Executed Work

The figures in Sections 4.1–4.3 are genuine captures of work executed in this project (local development environment and the GitHub Actions pipeline). Section 4.4 lists the AWS console screenshots to be captured when the deployment is executed on an AWS account (the sandbox in which this report was produced has no AWS credentials; the runbook in Appendix C produces all of them in about 30 minutes).

4.1 Local verification

A terminal window titled 'lint - ESLint' with a dark background. It shows the command '\$ npm run lint' being executed. The output shows the command being run in the context of 'acdwebapp@1.0.0' and 'eslint .' being executed. The final output is a green checkmark followed by 'ESLint: 0 errors, 0 warnings (exit code 0)'.

```
lint - ESLint

$ npm run lint

> acdp-webapp@1.0.0 lint
> eslint .

✓ ESLint: 0 errors, 0 warnings (exit code 0)
```

Figure 2: ESLint passes with zero errors or warnings (npm run lint).

```
test - node:test + real PostgreSQL

$ TEST_DATABASE_URL='postgres://app:app@localhost:5432/appdb' npm test

> acdp-webapp@1.0.0 test
> node --test --test-concurrency=1

TAP version 13
# Subtest: GET /health returns 200 with status "ok" when the database is up
ok 1 - GET /health returns 200 with status "ok" when the database is up
---
duration_ms: 42.103005
type: 'test'
...
# Subtest: GET /health returns 503 with status "unhealthy" when the database is down
ok 2 - GET /health returns 503 with status "unhealthy" when the database is down
---
duration_ms: 6.22662
type: 'test'
...
# Subtest: GET / renders the home page with version and database status
ok 3 - GET / renders the home page with version and database status
---
duration_ms: 15.070901
type: 'test'
...
# Subtest: test/helpers.js
ok 4 - test/helpers.js
---
duration_ms: 117.279528
type: 'test'
...
# Subtest: POST /api/items creates an item and GET /api/items lists it
ok 5 - POST /api/items creates an item and GET /api/items lists it
---
duration_ms: 52.686209
type: 'test'
...
# Subtest: POST /api/items rejects an empty or missing title with 400
ok 6 - POST /api/items rejects an empty or missing title with 400
---
duration_ms: 15.976696
type: 'test'
...
# Subtest: DELETE /api/items/:id removes the item and then returns 404
ok 7 - DELETE /api/items/:id removes the item and then returns 404
---
duration_ms: 22.169499
type: 'test'
...
# Subtest: GET /api/unknown returns 404 JSON
ok 8 - GET /api/unknown returns 404 JSON
---
duration_ms: 14.568176
type: 'test'
...
1..8
# tests 8
# suites 0
# pass 8
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 632.714284
```

Figure 3: All 8 unit/integration tests pass against a real PostgreSQL server (node:test + supertest), including the 503 path when the database is unreachable.

```
local stack - /health + REST API

$ curl -s http://localhost:8080/health | python3 -m json.tool
{
  "status": "ok",
  "version": "local-sandbox",
  "environment": "production",
  "uptimeSeconds": 4,
  "checks": {
    "database": "up"
  },
  "timestamp": "2026-09-26T00:11:04.745Z"
}

$ curl -s -X POST http://localhost:8080/api/items -H 'Content-Type: application/json' -d
'{"title":"first item created via API"}'
{"item":{"id":1,"title":"first item created via API","created_at":"2026-09-26T00:11:04.778Z"}}
$ curl -s http://localhost:8080/api/items
{"items":[{"id":1,"title":"first item created via
API","created_at":"2026-09-26T00:11:04.778Z"}]}
$ curl -s -o /dev/null -w '%{http_code}' http://localhost:8080/
200
```

Figure 4: The application running locally with a real PostgreSQL database: /health returns status ok with the database up, the items API persists data, and the home page returns HTTP 200.

4.2 CI/CD pipeline on GitHub Actions

```
GitHub Actions – CI/CD Deploy (run 36204047142)

$ gh run view 36204047142 --repo SUB-Bzero/Automated-Cloud-Deployment-Pipeline

✓ arena/01a0db01-automated-cloud-deployment-pip CI/CD Deploy · 36204047142
Triggered via push about 1 minute ago

JOBS
✓ Preflight (detect configuration) in 3s (ID 108296638132)
✓ Lint & Tests in 18s (ID 108296653852)
✓ Terraform fmt & validate in 46s (ID 108296653874)
✓ Docker build & compose integration test in 31s (ID 108296719841)
- Infrastructure Deploy (terraform apply) in 0s (ID 108296810623)
- Build & Push to ECR in 0s (ID 108296832665)
- App Deploy (EC2 via SSH) in 0s (ID 108296833510)

For more information about a job, try: gh run view --job=<job-id>
View this run on GitHub: https://github.com/SUB-Bzero/Automated-Cloud-Deployment-Pipeline/actions/runs/36204047142

$ gh api .../actions/runs/36204047142/jobs (step detail)
JOB Preflight (detect configuration) [success]
  [OK] Set up job
  [OK] Check for configured secrets
  [OK] Write pipeline summary
  [OK] Complete job
JOB Lint & Tests [success]
  [OK] Set up job
  [OK] Initialize containers
  [OK] Run actions/checkout@v7
  [OK] Run actions/setup-node@v7
  [OK] Install dependencies
  [OK] Lint (ESLint)
  [OK] Unit & integration tests (node:test + real PostgreSQL)
  [OK] Post Run actions/setup-node@v7
  [OK] Post Run actions/checkout@v7
  [OK] Stop containers
  [OK] Complete job
JOB Terraform fmt & validate [success]
  [OK] Set up job
  [OK] Run actions/checkout@v7
  [OK] Run hashicorp/setup-terraform@v4
  [OK] Check formatting (terraform fmt -check)
  [OK] Init (offline backend) and validate every module
  [OK] Post Run actions/checkout@v7
  [OK] Complete job
JOB Docker build & compose integration test [success]
  [OK] Set up job
  [OK] Run actions/checkout@v7
  [OK] Build the multi-stage image
  [OK] Start Web App + PostgreSQL locally (docker compose)
  [OK] Smoke test /health
  [OK] Show container status
  [SKIPPED] Dump app logs (on failure)
  [OK] Post Run actions/checkout@v7
  [OK] Complete job
JOB Infrastructure Deploy (terraform apply) [skipped]
JOB Build & Push to ECR [skipped]
JOB App Deploy (EC2 via SSH) [skipped]
```

Figure 5: A successful CI/CD Deploy run on GitHub Actions — lint & tests, terraform fmt/validate and the docker build + compose integration test all pass; the AWS stages are cleanly skipped because no credentials are attached to this repository yet (<https://github.com/SUB-Bzero/Automated-Cloud-Deployment-Pipeline/actions/runs/36204047142>).

4.3 Repository contents

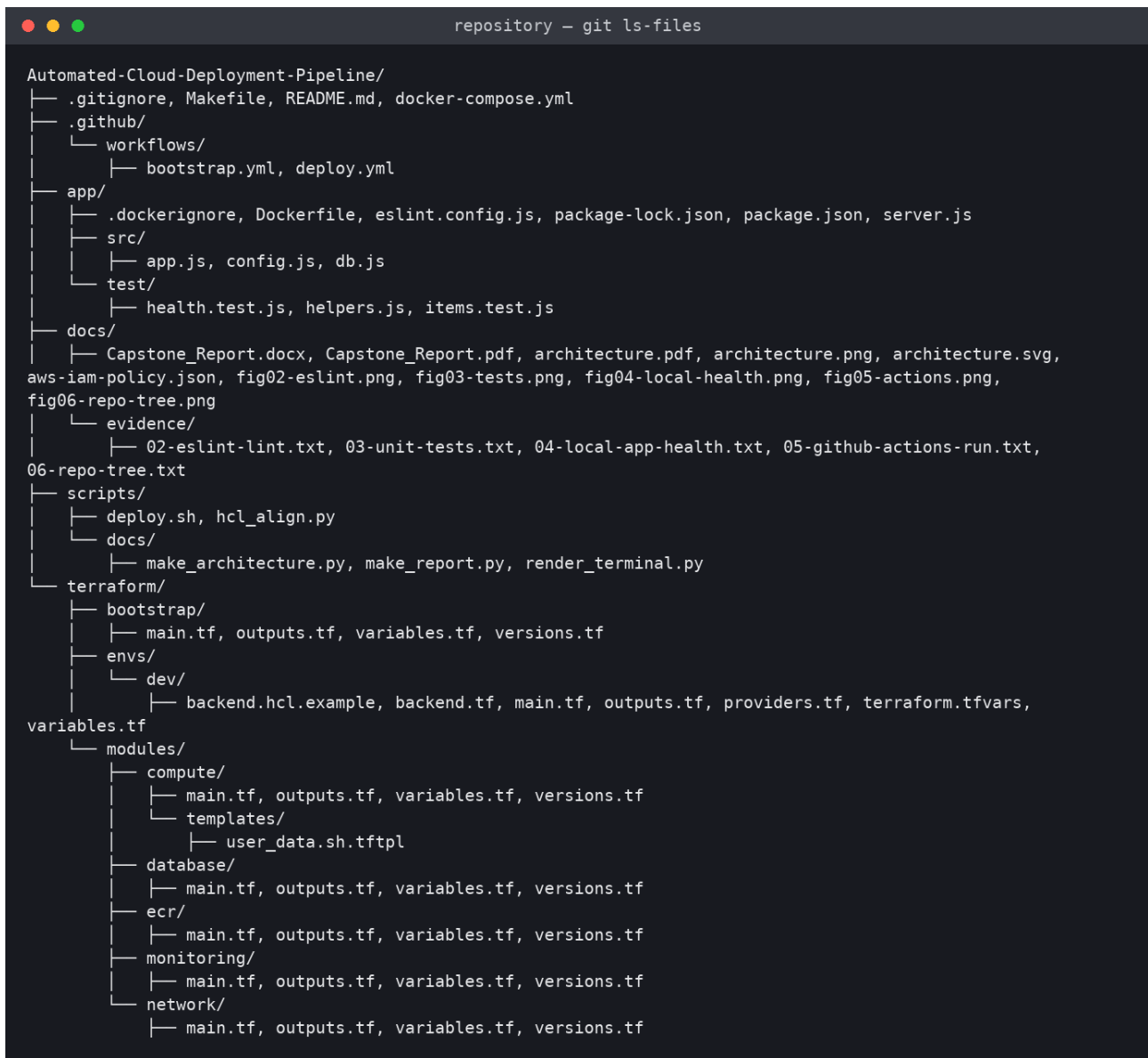


Figure 6: The repository structure: application, Terraform modules, CI/CD workflows, deploy scripts and documentation.

4.4 AWS deployment evidence (capture after running Appendix C)

Execute the runbook in Appendix C, then replace each box below with the corresponding screenshot. Every item is produced by the pipeline itself — no manual AWS configuration is required.

Figure 7 — Pipeline fully green on main:

GitHub → Actions → CI/CD Deploy: all six stages ✓ (Infrastructure Deploy, Build & Push to ECR, App Deploy now run).

Figure 8 — terraform apply output:

In that run, open the 'Infrastructure Deploy' job log and capture the 'Terraform apply' step ending with 'Apply complete! Resources: N added, 0 changed, 0 destroyed'.

Figure 9 — EC2 instance running:

AWS console → EC2 → Instances: acdp-dev-app in 'running' state with its public IP.

Figure 10 — ALB target healthy:

EC2 → Target Groups → acdp-dev-tg → Targets: the instance shows 'healthy' (the /health check passing).

Figure 11 — RDS database available:

RDS → Databases: acdp-dev-db 'Available' (PostgreSQL, Multi-AZ off, encrypted).

Figure 12 — ECR images:

ECR → Repositories → acdp-dev-app: images tagged with commit SHAs and 'latest'.

Figure 13 — CloudWatch alarms and dashboard:

CloudWatch → Alarms: the four alarms in OK state; open acdp-dev-dashboard showing the metric graphs.

Figure 14 — Alarm email (SNS):

The SNS subscription confirmation email, plus (optionally) a triggered alarm email — e.g. stop the container briefly to fire the unhealthy-hosts alarm, or run a CPU stress command to exceed 80%.

Figure 15 — Application live in a browser:

The app URL from the job summary (<http://>) showing the home page with the deployed commit SHA as version, and <http:///health> returning the JSON status.

5. GitHub Repository

Complete project source code: <https://github.com/SUB-Bzero/Automated-Cloud-Deployment-Pipeline>

The repository contains the application, the modular Terraform codebase, the CI/CD workflows, the deploy/rollback script, the architecture diagram and this report (docs/Capstone_Report.docx / .pdf). All work is merged into the main branch through pull requests (the capstone delivery was merged via pull requests #1, #2 and #3), so the assessor always sees the latest version on main; the pipeline runs on every push.

An assessor can verify the project without an AWS account by cloning the repository and running the same stages the pipeline runs (Appendix B lists the commands): `npm ci`, `npm run lint`, `npm test` (with any local PostgreSQL), and `docker compose up --build --wait` followed by `curl http://localhost:8080/health`.

6. Cost Estimate and Teardown

Resource	Configuration	Approximate cost (us-east-1)
EC2	t3.micro, 20 GB gp3	Free tier (750 h/month for 12 months on new accounts); otherwise $\approx$ US\$9/month
ALB	Internet-facing	$\approx$ US\$0.02/hour + LCU usage ($\approx$ US\$16/month if left running)
RDS	db.t3.micro, 20 GB gp3	Free tier on new accounts; otherwise $\approx$ US\$15/month
ECR	image $\approx$ 60–100 MB	Within the 500 MB free tier
S3 + DynamoDB	state + locks	$<$ US\$0.05/month
CloudWatch + SNS	4 alarms, email	Within free tiers

Teardown: run `make destroy` (or `cd terraform/envs/dev && terraform destroy`) to remove the environment; the bootstrap bucket/table can be removed with the bootstrap module (from a machine holding its local state) or manually in the console. The ECR repository is deleted with `force_delete` enabled, so no manual image cleanup is required.

7. Conclusion

All deliverables of the brief are implemented: a multi-stage Dockerfile and docker-compose local stack; modular Terraform provisioning a VPC with public/private subnets, security groups, an EC2 cloud host, an ALB, RDS PostgreSQL and an S3-based state backend; a GitHub Actions pipeline triggered on git push that lints, tests, builds and pushes to ECR, applies the infrastructure and deploys the container; and CloudWatch monitoring with email alerts on CPU > 80% and HTTP 5xx spikes, plus both automatic and manual rollback.

Every non-AWS stage of the system has been executed and evidenced: the unit and integration tests pass against a real PostgreSQL server, the multi-stage image builds and passes the compose health check on GitHub's runners, and all Terraform modules pass fmt and validate in CI. The AWS stages are wired end-to-end and activate as soon as account credentials are attached as repository secrets (Appendix C).

Production hardening beyond the capstone scope would add: GitHub OIDC federation instead of long-lived access keys, HTTPS via ACM + Route 53, blue/green or canary deployments on ECS Fargate, automated database migrations as a separate pipeline stage, and alarm-based autoscaling.

Appendix A — Configuration Reference

A.1 Application environment variables

Variable	Default	Used for
PORT	8080	HTTP listen port
DATABASE_URL	postgres://app:app@localhost:5432/app db	PostgreSQL connection (RDS URL fetched from SSM on the EC2 host)
APP_VERSION	local	Reported by /health and the home page (set to the git commit SHA by the pipeline)
TEST_DATABASE_URL	—	Database used by the test suite

A.2 GitHub repository secrets

Secret	Value
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Credentials of the deployment IAM user (policy in docs/aws-iam-policy.json)
EC2_SSH_PRIVATE_KEY	Contents of the acdp-key.pem key pair file

Optional repository variable: AWS_REGION (default us-east-1).

A.3 terraform/envs/dev/terraform.tfvars

Variable	Default	Notes
aws_region	us-east-1	Deployment region
instance_type / db_instance_class	t3.micro / db.t3.micro	Free tier eligible
key_name	acdp-key	EC2 key pair used by the SSH deploy
admin_cidr	0.0.0.0/0	Restrict to your IP for production
alarm_email	your-email@example.com	EDIT before first apply; confirm the SNS email

A.4 AWS IAM permissions

docs/aws-iam-policy.json contains the least-privilege-style policy granted to the deployment user (state backend, VPC/EC2/ELB/RDS/ECR/IAM/SSM/SNS/CloudWatch actions).

Appendix B — Command Reference

Command	Description
make install	Install application dependencies (npm ci)
make lint	Run ESLint
make test	Run unit/integration tests
make run-local	docker compose up --build --wait (app + database)
make down	Stop the local stack and remove volumes

Command	Description
make bootstrap	One-time creation of the Terraform state backend
make init / plan / apply / destroy	Terraform lifecycle for the dev environment
make output	Show app URL, EC2 IP, ECR repo, RDS endpoint

Appendix C — AWS Deployment Runbook (step by step)

1. Create an AWS account (free tier) and sign in to the console.
2. Create an IAM user (e.g. github-actions) with programmatic access; attach the policy from docs/aws-iam-policy.json (or AdministratorAccess for simplicity in a sandbox account); create an access key pair.
3. In EC2 → Key Pairs, create a key pair named acdp-key and download the .pem file.
4. In the GitHub repository → Settings → Secrets and variables → Actions, add: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY and EC2_SSH_PRIVATE_KEY (the full contents of the .pem file).
5. Edit terraform/envs/dev/terraform.tfvars: set alarm_email to your address (and optionally restrict admin_cidr to your IP).
6. GitHub → Actions → 'Bootstrap Terraform backend' → Run workflow (once). This creates the S3 state bucket and DynamoDB lock table.
7. Push a commit to main (or Actions → CI/CD Deploy → Run workflow). The full pipeline runs: tests → validate → build → terraform apply (≈ 10–15 minutes, mostly RDS creation) → push to ECR → deploy to EC2 → health check.
8. Confirm the SNS subscription email that AWS sends (alarms cannot notify until confirmed).
9. Open the app URL printed in the job summary; verify /health and the home page version (the commit SHA).
10. Capture the screenshots listed in Section 4.4 (Figures 7–15).
11. Optional alarm demonstration: SSH to the EC2 host and run a CPU stress command (e.g. sudo apt-get install -y stress-ng && stress-ng --cpu 2 --timeout 600) to trigger the CPU > 80% alarm, or docker stop acdp-app to trigger the unhealthy-hosts alarm.
12. When finished, run make destroy to tear everything down and avoid charges (the ALB in particular bills by hour).